KodetoCareer — Master Prompt for AI Coding Agent

Version 1.0 | Use with Claude, GPT-4o, or Cursor AI

SYSTEM CONTEXT (Paste this at the start of every AI coding session)

```
You are a senior full-stack engineer building the KodetoCareer Training & Placement Man...

PRODUCT SUMMARY:
KodetoCareer is a platform for a training company that upskills students at partner col...

TECHNICAL STACK:
- Backend: Node.js 20, Express.js 4, TypeScript (strict), Prisma 5 ORM, PostgreSQL 15, ...
- Frontend Web (Admin): React 18, Vite, TypeScript, Tailwind CSS, shadcn/ui, React Quer...
- Mobile: Flutter 3, Riverpod 2, Dio, go_router
- Storage: AWS S3 + CloudFront CDN
- Video: AWS MediaConvert (HLS transcoding)
- Auth: JWT (RS256) with refresh token rotation
- Push: Firebase Cloud Messaging (FCM)
- Email: AWS SES with Nodemailer
- SMS: MSG91
- Queue: Bull (Redis-backed)

DATABASE: PostgreSQL with Prisma ORM. Multi-tenancy enforced at application layer via c...

API STANDARDS:
- RESTful, base URL: /v1/
- All responses use envelope: { success: boolean, data?: any, error?: { code, message }...
- Auth via Bearer token in Authorization header
- All inputs validated with Zod before controller logic
- Role-based access: SUPER_ADMIN > COLLEGE_ADMIN > TRAINER > STUDENT

CODE PRINCIPLES:
1. Controller layer: HTTP only (parse request → call service → return response)
2. Service layer: Business logic only (no HTTP concerns, no direct DB in controllers)
3. Use Prisma for all database operations — no raw SQL unless absolutely required
4. All write operations auto-logged in audit_logs table
5. Never expose raw error messages to client — always use AppError class
6. All file uploads go through presigned S3 URLs
7. Background jobs (email, PDF gen, video processing) always go through Bull queues
8. TypeScript strict mode — no 'any' types

FOLDER STRUCTURE (Backend):
src/
```

```
  config/ — database, redis, aws, email setup
  middleware/ — auth, rbac, rateLimiter, errorHandler, auditLog, validate
  modules/ — one folder per domain (auth, colleges, students, courses, etc.)
    Each module: {name}.router.ts, {name}.controller.ts, {name}.service.ts, {name}.sche...
  jobs/ — Bull workers (video, certificate, email, sms, report)
  services/ — shared services (storage, email, sms, fcm, pdf, analytics)
  utils/ — response helpers, pagination, codeGenerator, csvParser, logger
  index.ts — entry point

FLUTTER STRUCTURE:
lib/
  core/ — api client, secure storage, theme, router, utils
  features/ — one folder per feature (auth, dashboard, courses, quizzes, etc.)
    Each feature: data/ (repository, api), domain/ (models), presentation/ (screens, pr...
  shared/ — shared widgets, models

REACT STRUCTURE:
src/
  components/ — ui/ (shadcn), common/ (shared app components), layout/
  pages/ — one folder per domain
  hooks/ — React Query hooks per domain
  services/ — Axios API calls per domain
  store/ — Zustand stores (authStore)

CURRENT USER ROLE CONTEXT: [SPECIFY THE ROLE WHEN PROMPTING — e.g., "Super Admin admin ...
```

MODULE-SPECIFIC PROMPTS

Copy the system context above PLUS the relevant module prompt below for each coding session.

PROMPT 1: Authentication System

```
BUILD: Complete authentication system for KodetoCareer.

REQUIRED ENDPOINTS:
1. POST /v1/auth/register — Student self-registration
   Input: { firstName, lastName, email, password }
   Logic: Hash password (bcrypt 12 rounds), create user (role: STUDENT), generate 6-dig...

2. POST /v1/auth/verify-email — Verify OTP
   Input: { email, otp }
   Logic: Find unexpired unused OTP for email, mark used, set user.isEmailVerified=true...

3. POST /v1/auth/login — Login for all roles
   Input: { email, password }
   Logic: Find user by email (must be active, not soft-deleted), compare password, if s...
4. POST /v1/auth/refresh — Rotate refresh token
```

```
   Input: refresh token in httpOnly cookie OR request body (for mobile)
   Logic: Find valid unexpired token by hash, verify not revoked, generate new access +...

5. POST /v1/auth/forgot-password — Send reset OTP
   Input: { email }
   Logic: Find user, generate OTP (type: PASSWORD_RESET), send email, return success (s...

6. POST /v1/auth/reset-password — Reset with OTP
   Input: { email, otp, newPassword }
   Logic: Validate OTP, hash new password, update user, invalidate all existing refresh...

7. POST /v1/auth/logout — Invalidate session
   Auth required. Logic: Revoke current refresh token.

ADMIN CREATION (separate from self-registration):
- Super Admin creates College Admin and Trainer accounts via /v1/college-admins and /v1...
- System generates temporary password, sends welcome email with credentials
- First login triggers forced password change (flag: requiresPasswordChange in users ta...

TOKEN SPECS:
- Access token: RS256 JWT, payload: { userId, role, collegeId (for college admin), exp:...
- Refresh token: 64-byte random hex, stored as SHA256 hash in refresh_tokens table, exp...
- Mobile: both tokens in response body. Web: refresh token in httpOnly Secure SameSite=...

RATE LIMITING: Login endpoint max 5 attempts per IP per 15 minutes. Use ioredis for sto...

FLUTTER IMPLEMENTATION:
- auth_provider.dart using AsyncNotifier<AuthState>
- AuthState: { user?, isLoading, error?, isAuthenticated }
- Dio interceptor: on 401 response → auto-call refresh → retry original request → if re...
- Tokens stored in flutter_secure_storage

DELIVER:
- Complete router, controller, service, and Zod schemas for backend
- AuthRepository and AuthNotifier for Flutter
- AuthProvider and useAuth hook for React
- All email templates referenced (welcome, otp, password-reset)
```

PROMPT 2: Course Builder (Admin Panel)

```
BUILD: Course management system — admin panel (React) + API (Node.js).

COURSE HIERARCHY: Course → Modules → Lessons
A lesson can have: video (upload or URL), PDF notes, rich text content, external links.

ADMIN PANEL REACT COMPONENTS:

1. CourseListPage
   - Table with columns: Title, Category, Difficulty, Status (badge), Lessons count, Cr...
   - Status filter tabs: All | Draft | Published
   - Search by title
   - [+ Create Course] button → opens CourseCreateModal
```

```
   - Row click → navigate to CourseBuilderPage

2. CourseCreateModal (slide-in panel, not full page)
   Fields: Title*, Slug (auto-generated from title, editable), Category (dropdown), Dif...
   [Cancel] [Save as Draft]
   On save → navigate to CourseBuilderPage/{courseId}

3. CourseBuilderPage (main editor)
   Left panel (300px): Module list (draggable to reorder)
     - Each module: collapse/expand, drag handle, module title (inline edit on click), ...
     - [+ Add Module] button at bottom

   Right panel: Lesson editor (shows when lesson selected)
     - Lesson title (inline edit)
     - Tab 1: Content
       - Lesson type selector (VIDEO | PDF | TEXT | EXTERNAL_LINK)
       - VIDEO: Two sub-tabs: [Upload MP4] and [Paste URL (YouTube/Vimeo)]
         - Upload: file picker, shows upload progress bar, then processing status
       - PDF: multi-file upload (up to 5 PDFs per lesson)
       - TEXT: TipTap rich text editor
       - EXTERNAL_LINK: URL input + display title input
     - Tab 2: Settings
       - Is Mandatory toggle
       - Is Preview (viewable without login) toggle
       - Duration (minutes, optional)

   Top bar: Course title, status badge, [Save Draft] [Publish Course] buttons

4. Lesson ordering: drag-and-drop within a module (dnd-kit library)
5. Module ordering: drag-and-drop in the left panel

API ENDPOINTS NEEDED:
GET    /v1/courses                    — list with pagination
POST   /v1/courses                    — create course
GET    /v1/courses/:id                — get full course (with modules and lessons)
PUT    /v1/courses/:id                — update course
POST   /v1/courses/:id/publish        — set status to PUBLISHED
DELETE /v1/courses/:id                — soft delete
POST   /v1/courses/:id/modules        — add module
PUT    /v1/modules/:id                — update module (title, order)
DELETE /v1/modules/:id                — delete module
POST   /v1/modules/:id/lessons        — add lesson
PUT    /v1/lessons/:id                — update lesson (all fields)
DELETE /v1/lessons/:id                — delete lesson
PUT    /v1/courses/:id/module-order   — reorder modules { moduleIds: string[] }
PUT    /v1/modules/:id/lesson-order   — reorder lessons { lessonIds: string[] }
GET    /v1/uploads/presign            — get presigned S3 URL for direct upload
POST   /v1/uploads/confirm            — confirm upload complete, trigger processing

DELIVER: Complete React pages and components + backend endpoints with Prisma queries.
```

PROMPT 3: Attendance System

```
BUILD: Complete attendance management — trainer interface (React) + student view (Flutt...

TRAINER ATTENDANCE FLOW (React):

1. TrainerDashboard shows today's sessions:
   Batch Name | Time | Topic | [Mark Attendance] button
   If already marked: [View/Edit] button

2. AttendanceMarkingPage
   Route: /trainer/sessions/:sessionId/attendance

   Header section:
   - Batch name, date, session time
   - Topic covered (editable text input — trainer fills in what was covered)
   - Session mode toggle: Online / Offline
   - Recording URL field (optional)

   Student list (sorted by name alphabetically):
   - Student name, photo thumbnail (or initials avatar)
   - Three-button toggle: [P] Present | [A] Absent | [L] Late
   - Default: Absent (safest default, trainer must actively mark Present)
   - Note field (expands on hover/click for Late students)

   Bulk actions bar at top:
   - [✓ Mark All Present] button
   - Live count: "Present: 45 | Absent: 12 | Late: 3"

   [Submit Attendance] button (sticky at bottom)
   On submit: confirmation dialog showing summary → POST to API → success toast

3. EditAttendancePage (within 24 hours of session)
   Same UI but shows existing marks pre-filled
   Super Admin can access this anytime (no time restriction)
   Changes logged in audit_logs

API ENDPOINTS:
POST   /v1/sessions                    — Create class session
GET    /v1/sessions/batch/:batchId     — Get all sessions for a batch
GET    /v1/sessions/:id                — Get session with attendance records
POST   /v1/sessions/:id/attendance     — Submit attendance (body: { records: [{ student...
PUT    /v1/sessions/:id/attendance     — Edit existing attendance
GET    /v1/attendance/student/:studentId/batch/:batchId — Student's attendance summary ...
GET    /v1/attendance/batch/:batchId/export — Download Excel attendance report

BUSINESS LOGIC:
- When attendance submitted: calculate attendance % for each student
- If student attendance < 75%: create notification for student AND log in analytics
- Attendance edit allowed within 24h by trainer, anytime by super admin
- All edits logged in audit_logs with old_values and new_values
- Attendance export: Excel file with columns: Name | Enrollment No | Each Session Date ...

FLUTTER STUDENT VIEW:
- AttendanceScreen in Profile or dedicated tab
- Card per batch: Batch name, attendance percentage (circular indicator), sessions atte...
- Expandable list of individual sessions: Date | Topic | Status (Present/Absent/Late)
```

```
- Alert widget if attendance < 75%: "■■ Your attendance is below the required 75% for c...

DELIVER: Complete implementation for all three (React trainer, Flutter student, Node.js...
```

PROMPT 4: Quiz Engine

```
BUILD: Complete quiz engine — admin creation (React) + student quiz-taking (Flutter) + ...

QUIZ CREATION (React Admin):

QuizBuilderPage with sections:
1. Quiz Settings (top form):
   - Title*, Course (select), Module (select, filtered by course), Batch assignment (op...
   - Time Limit (minutes, optional — unchecked = no limit)
   - Total marks (auto-calculated from questions), Pass marks
   - Attempts allowed (1–5)
   - Shuffle questions toggle, Shuffle options toggle
   - Show answers immediately after submit toggle
   - Available From / Until datetime pickers
   [Save Settings]

2. Questions Section:
   - Question list (numbered, drag to reorder)
   - [+ Add Question] button
   - Add Question form (inline, below list):
     - Question type selector: MCQ Single | MCQ Multiple | True/False | Short Text
     - Question text (rich text for code snippets)
     - Marks value (default 1)
     - Difficulty: Easy | Medium | Hard
     - Topic tag (text input)
     - Options section (for MCQ/True-False):
       - Option inputs with [Correct] toggle
       - MCQ Single: radio-style — selecting one deselects others
       - MCQ Multiple: checkbox-style — multiple can be marked correct
     - Explanation field (shown to students after attempt)
   [Add Question] / [Save Changes]

3. Preview tab — shows quiz exactly as students see it

STUDENT QUIZ FLOW (Flutter):

QuizListScreen:
- Filter tabs: Upcoming | Active | Completed
- Quiz card: title, course, available until, time limit, status badge

QuizDetailScreen:
- Quiz info: questions count, time limit, marks, attempts
- If available and not started: [Start Quiz] button
- If in progress (auto-saved): [Resume]
- If completed: score summary, [View Results]

Pre-quiz bottom sheet:
```

```
- Rules reminder: time limit, attempts remaining
- [Cancel] [Begin Quiz]

QuizScreen (stateful, critical):
- Single question per page (not all scrollable)
- Top: Question X of Y | Timer (HH:MM:SS) — red when < 5min
- Progress dots (tap to jump to any question — MCQ style)
- For MCQ Single: radio button options
- For MCQ Multiple: checkbox options
- For True/False: two large buttons True/False
- [Previous] [Next] navigation
- [Submit Quiz] button (always visible)
- Auto-save state to server every 30 seconds: PUT /v1/quiz-attempts/:id/save
  Payload: { answers: [{ questionId, selectedOptionIds?, textAnswer? }] }
- If network lost: save to local SQLite, sync on reconnect
- On time expiry: auto-submit

Submit confirmation bottom sheet:
- Shows: answered X / total Y, unanswered Z
- [Review] [Submit Now]

QuizResultScreen:
- Score: X/Y (large, prominent)
- Passed/Failed badge (green/red)
- Stat row: Correct | Wrong | Skipped | Time Taken
- Rank in batch: #N
- [View Answer Sheet] [See Leaderboard]

AnswerReviewScreen:
- Each question listed with:
  - Question text
  - Student's answer (highlighted green if correct, red if wrong)
  - Correct answer(s) shown
  - Explanation text

API ENDPOINTS:
POST   /v1/quizzes                      — Create quiz
GET    /v1/quizzes/:id                  — Get quiz (without correct answers for students)
POST   /v1/quizzes/:id/questions        — Add question
PUT    /v1/quiz-questions/:id           — Update question
DELETE /v1/quiz-questions/:id           — Delete question
POST   /v1/quizzes/:id/start            — Start attempt, returns attempt ID
PUT    /v1/quiz-attempts/:id/save       — Auto-save progress
POST   /v1/quiz-attempts/:id/submit     — Submit attempt, returns score
GET    /v1/quiz-attempts/:id/result     — Get result with answers (after submission)
GET    /v1/quizzes/:id/leaderboard      — Top 10 scores in batch
GET    /v1/quizzes/:id/analytics        — Admin: per-question stats

SCORING ENGINE (server-side):
For MCQ Single: selectedOptionId must match exactly one correct option_id
For MCQ Multiple: selectedOptionIds must match ALL correct option_ids AND no incorrect ...
  - Partial credit option (configurable per quiz): award proportional marks for partial...
For True/False: same as MCQ Single
For Short Text: marks = 0 by default (requires manual grading — flag as "Pending Review")

LEADERBOARD: Top 10 by score descending, then time_taken ascending (faster wins ties)
```

```
Refresh leaderboard Redis cache on each new submission.

DELIVER: Complete quiz engine implementation across all layers.
```

PROMPT 5: Certificate Generation Pipeline

```
BUILD: Certificate generation pipeline — admin trigger (React) + PDF generation (Node.j...

CERTIFICATE PDF GENERATION (Node.js with PDFKit):

Template spec:
- Page: A4 Landscape (841.89 × 595.28 points in PDFKit)
- Fonts: embed Inter-Bold.ttf for headings, Inter-Regular.ttf for body
- Gold decorative border (4-sided frame using lines/rectangles)
- Company logo top-left (load from S3 or local assets)
- Layout structure as specified in the Development Specification

Code structure:
certificates/
  certificate.worker.ts    — Bull job handler
  certificate.service.ts   — Orchestration
  certificate.generator.ts — PDFKit PDF creation
  certificate.uploader.ts  — S3 upload
  certificate.emailer.ts   — Send email with PDF attached

Generator function signature:
generateCertificate(data: {
  studentName: string,
  courseName: string,
  collegeName: string,
  completionDate: Date,
  certificateCode: string,
  trainerName: string,
  batchDuration: string,     // "4 Months"
  qrCodeDataUrl: string,     // pre-generated QR pointing to verify URL
  logoBase64: string
}): Promise<Buffer>   // Returns PDF buffer

QR Code generation: Use 'qrcode' npm package
QR content: https://verify.kodetocareer.com/{certificateCode}

ELIGIBILITY CHECK ENDPOINT:
GET /v1/certificates/eligibility/:batchId
Returns for each enrolled student:
{
  studentId, studentName, enrollmentNumber,
  attendancePct, quizAvgPct, assignmentsComplete, assignmentsTotal,
  isEligible,
  ineligibilityReasons: string[],   // ["Attendance below 75%", "2 assignments missing"]
  hasOverride: boolean,
  overrideReason: string | null
}
```

```
BULK GENERATION ENDPOINT:
POST /v1/certificates/generate
Body: { batchId, studentIds: string[], overrides?: { studentId, reason }[] }
Auth: SUPER_ADMIN only

Flow:
1. Validate all studentIds are in the batch
2. Apply eligibility check (skip if override provided)
3. Create Bull jobs for each certificate
4. Return { jobGroupId, totalCount, studentIds }

Bull Worker per certificate:
1. Fetch student data, course data, college data, batch data
2. Generate unique certificate code: KTC-CERT-{YEAR}-{5-DIGIT-RANDOM}
3. Generate QR code DataURL
4. Generate PDF using generator
5. Upload PDF to S3: certificates/{year}/{studentId}/{certificateCode}.pdf
6. Create certificate record in DB
7. Update student placement records if needed
8. Send email to student with PDF attached
9. Send push notification to student
10. Emit socket event for admin progress tracking

Progress tracking (Socket.io):
Admin page subscribes to: /certificates/{batchId}
Events:
  certificate:progress — { completed: n, total: n, studentId, status: 'SUCCESS' | 'FAIL...
  certificate:complete — { completed: n, failed: m, downloadUrl: '...' }

Batch download: Generate ZIP of all PDFs, upload to S3 with 1-hour expiry presigned URL

PUBLIC VERIFICATION:
GET /v1/certificates/verify/:code (NO AUTH required)
Returns: { valid: true, studentName, courseName, issuedAt, collegeName } or { valid: fa...
This endpoint is called by the QR code scan → shows a simple verification page

REACT ADMIN COMPONENT (CertificateManagementPage):
1. Batch selector at top
2. Eligibility table: Name | Enroll No | Attend% | Quiz Avg | Assignments | Eligible (■...
3. Override modal: reason text required
4. Filter: Show Eligible | Show Ineligible | Show All
5. [Generate for Eligible (N)] button
6. Progress modal (socket-connected):
   - Progress bar
   - Live count "Generated: X / Y"
   - Failure list with reasons
   - [Download All (ZIP)] when complete

FLUTTER CERTIFICATE SCREEN:
- MyCertificatesScreen: list of earned certificates
- CertificateViewerScreen: full-screen PDF viewer using flutter_pdfview
- Share options: native share sheet + specific LinkedIn share
  LinkedIn share: open browser to linkedin.com/sharing/share-offsite/?url={cert_verify_...
- Download button: save PDF to device downloads folder

DELIVER: Complete certificate pipeline across all layers with all edge cases handled.
```

PROMPT 6: Analytics Dashboard

```
BUILD: Analytics dashboards — Super Admin and College Admin levels (React + API).

SUPER ADMIN DASHBOARD:

TopRow KPI cards (auto-refresh every 5 minutes):
1. Total Active Students (count where batch_students.status = ACTIVE)
2. Total Active Colleges (count where colleges.is_active = true AND has active batch)
3. Students Placed This Month (count placement_records where offer_date in current month)
4. Average Attendance Platform-wide (avg of attendance_summary.attendance_pct)
5. Active Batches (count where batches.status = ACTIVE)
6. Certificates Issued (count certificates where is_valid = true)

Charts (using Recharts):
1. Monthly Enrollment Trend (LineChart):
   - X: Last 12 months
   - Y: New students enrolled that month
   - Query: GROUP BY DATE_TRUNC('month', batch_students.enrolled_at)

2. Placement by Company (BarChart, top 10):
   - X: Company names
   - Y: Number of students placed
   - Sorted descending by count

3. Attendance Distribution (DonutChart):
   - Segments: 90-100% | 75-90% | 60-75% | Below 60%
   - Query: bucket attendance_summary.attendance_pct

4. Course Completion Funnel (FunnelChart or horizontal bar):
   - Enrolled | Started | 50% Complete | 100% Complete

Students at Risk Table:
- Query: students with attendance_pct < 75 OR quiz_avg < 40 OR last activity > 14 days
- Columns: Name | College | Batch | Issue | Days Since Active | [Send Alert]
- Send Alert: creates an in-app notification to that student

COLLEGE ADMIN DASHBOARD:

Same structure but ALL queries scoped to the admin's college_id.

Additional:
- Comparison to platform average (subtle grey benchmark line on charts)
- Top 5 performing students (by placement_readiness_score)
- Download buttons: [Attendance Report PDF] [Progress Report Excel] [Placement Report PDF]

REPORT GENERATION (async, Bull queue):

ReportTypes:
1. ATTENDANCE_REPORT — for a batch + date range → Excel with one row per student + sess...
2. PROGRESS_REPORT — for a college → Excel with student completion stats
```

```
3. PLACEMENT_REPORT — for a college → PDF/Excel with placement data

POST /v1/reports/generate
Body: { type, filters: { batchId?, collegeId?, dateFrom?, dateTo? } }
Returns: { reportId } (job queued)

GET /v1/reports/:reportId/status
Returns: { status: 'PENDING' | 'PROCESSING' | 'READY' | 'FAILED', downloadUrl? }

Report download URLs are presigned S3 URLs with 1-hour expiry.

API ENDPOINTS:
GET /v1/analytics/dashboard          — Super admin summary metrics
GET /v1/analytics/college/:id        — College-specific metrics (scoped for college admin)
GET /v1/analytics/enrollment-trend   — Monthly enrollment line chart data
GET /v1/analytics/placement-by-company — Bar chart data
GET /v1/analytics/attendance-distribution — Donut chart data
GET /v1/analytics/students-at-risk   — Risk report
GET /v1/analytics/top-performers     — Top N students by readiness score
POST /v1/reports/generate            — Queue report generation
GET /v1/reports/:id/status           — Check report status

CACHING: All analytics endpoints cached in Redis for 30 minutes.
Cache invalidation: on batch_students change, attendance_records change.

DELIVER: Complete React analytics pages with recharts + API endpoints with optimised SQ...
```

PROMPT 7: Mobile App Shell (Flutter)

```
BUILD: Flutter app shell with navigation, theming, and core infrastructure.

APP SHELL REQUIREMENTS:

1. main.dart — App entry point
   - Initialize Firebase, Dio, secure storage
   - ProviderScope wrapping entire app
   - Handle FCM token registration on startup

2. app.dart — MaterialApp with go_router
   - Custom theme (colors, typography, button styles from design brief)
   - Router configuration with redirect logic:
     * If no token → /login
     * If token + profile incomplete → /profile-setup
     * Otherwise → /dashboard (bottom nav root)

3. Bottom Navigation (5 tabs):
   Home (/dashboard) | Learn (/courses) | Tests (/quizzes) | Placement (/placement) | P...
   - go_router ShellRoute for bottom nav persistence
   - Notification badge on Home tab for unread count

4. App Theme (app_theme.dart):
   Primary: Color(0xFF1E3A8A)
```

```
   Accent/Success: Color(0xFF10B981)
   Warning: Color(0xFFF59E0B)
   Error: Color(0xFFEF4444)
   Background: Color(0xFFF8FAFC)

   TextTheme based on Inter font family:
   - displayLarge: 28px, bold
   - headlineMedium: 22px, bold
   - titleLarge: 18px, semiBold
   - bodyLarge: 15px, regular
   - bodyMedium: 14px, regular
   - labelSmall: 12px, medium

5. Dio client setup (api_client.dart):
   - Base URL from env
   - Timeout: connect 10s, receive 30s
   - Request interceptor: add Authorization header
   - Response interceptor:
     * On 401: attempt token refresh → retry request → if refresh fails → logout
     * On network error: check connectivity, show appropriate error
   - Logging in debug mode

6. Shared Widgets to build:
   - PrimaryButton: full-width, loading state, disabled state
   - SecondaryButton: outlined style
   - CustomTextField: with label, error state, password toggle
   - SkeletonLoader: shimmer effect widget (use shimmer package)
   - ErrorStateWidget: icon + message + retry button
   - EmptyStateWidget: illustration + message + optional action button
   - ProgressBar: custom linear with colour and percentage label
   - Avatar: circular image with initials fallback
   - StatusBadge: coloured pill with text

7. Push Notification handling:
   - Background: go to relevant screen based on data.screen field
   - Foreground: show in-app banner (top overlay, 3-second auto-dismiss)
   - FCM token: get on app start, send to backend at POST /v1/users/me/fcm-token

8. Offline Detection:
   - connectivity_plus package
   - Show persistent top banner when offline: "No internet connection"
   - Banner dismisses automatically when connectivity restored

pubspec.yaml dependencies:
flutter_riverpod: ^2.4.0
go_router: ^13.0.0
dio: ^5.4.0
flutter_secure_storage: ^9.0.0
firebase_core: ^2.24.0
firebase_messaging: ^14.7.0
firebase_analytics: ^10.8.0
firebase_crashlytics: ^3.4.0
video_player: ^2.8.0
chewie: ^1.7.0
flutter_pdfview: ^1.3.2
sqflite: ^2.3.0
```

```
shimmer: ^3.0.0
connectivity_plus: ^5.0.2
image_picker: ^1.0.7
qr_flutter: ^4.1.0
cached_network_image: ^3.3.0
intl: ^0.19.0
lottie: ^2.7.0

DELIVER: Complete Flutter app shell with all infrastructure, theming, navigation, and s...
```

GENERAL CODING INSTRUCTIONS (Include with any prompt)

```
CODING STANDARDS TO FOLLOW:

1. TypeScript/Dart: Always use proper types. No 'any' in TypeScript. No dynamic in Dart...

2. Error handling:
   - Backend: throw AppError with appropriate status code and error code
   - Flutter: use Result<T, AppException> pattern or AsyncValue
   - React: React Query error state handling + toast notifications

3. Loading states: Every async operation must show a loading indicator. Never show a bl...

4. Empty states: Every list must have a designed empty state component.

5. Comments: Document WHY, not WHAT. Complex algorithms and business logic need comments.

6. Function size: If a function exceeds 50 lines, split it into smaller functions.

7. Naming:
   - Variables/functions: camelCase
   - Classes/types/interfaces: PascalCase
   - Constants: SCREAMING_SNAKE_CASE
   - Files: kebab-case (TS/React), snake_case (Dart)
   - Database columns: snake_case

8. Don't repeat yourself: If you write the same logic twice, extract to a shared utility.

9. Security: Never log sensitive data (passwords, tokens, PII). Use audit_logs for admi...

10. Test: For every function you write in service layer, consider the edge cases:
    - What if the related record doesn't exist?
    - What if the user doesn't have permission?
    - What if the input is at the boundary of validation?

OUTPUT FORMAT:
- Provide complete, working code files
- Include all imports
- Add TypeScript types/interfaces inline
- Add brief comments on complex sections
- If multiple files are needed, clearly label each file path
```

End of Master Prompt for AI Coding Agent

This document should be versioned alongside the codebase.